Cake Pattern

ke · Multi-layer repo→service→controller app; test cake swaps fak · Build with Wa

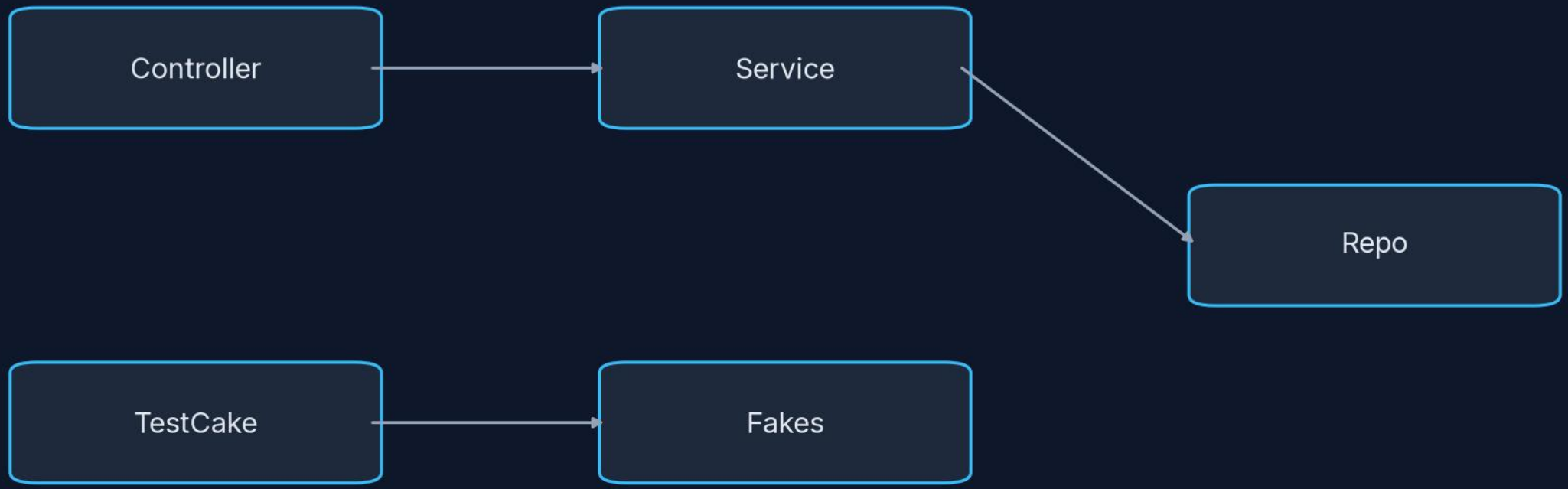
Today's Hook

- Problem: Multi-layer repo→service→controller app; test cake swaps fakes
- Pattern: Cake — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Multi-layer repo→service→controller app; test cake swaps fakes
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Cake: Multi-layer repo→service→controller app; test cake

Real-World Scenario

- Running example: Multi-layer repo→service→controller app; test cake swaps fakes
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

import java.util.LinkedHashMap;
import java.util.Map;

/** Naive: controller news service news repo — cannot swap fakes without rewriting. */
public final class NaiveApp {
    private final Map<String, String> db = new LinkedHashMap<>();
    public String postRegister(String id, String email) {
        if (db.containsKey(id)) return "409:exists";
        db.put(id, email);
        System.out.println("email " + email); // hard-wired
        return "201:" + id;
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Multi-layer cake: Repo → Service → Controller, with a test cake swapping fakes. */
public final class AppCake {
    private AppCake() {}

    public record User(String id, String email) {}

    public interface UserRepo {
        Optional<User> find(String id);
        User save(User user);
    }

    public interface EmailGateway {
        void send(String to, String body);
        List<String> sent();
    }
```

Solution (2)

```
public interface UserService {
    User register(String id, String email);
    String welcome(String id);
}

public interface UserController {
    String postRegister(String id, String email);
    String getWelcome(String id);
}

public interface UserRepoComponent { UserRepo userRepo(); }
public interface EmailComponent { EmailGateway emailGateway(); }
public interface UserServiceComponent extends UserRepoComponent, EmailComponent {
    default UserService userService() {
        return new UserService() {
            public User register(String id, String email) {
                var existing = userRepo().find(id);
                if (existing.isPresent()) throw new IllegalStateException("exists:" + id);
                var u = userRepo().save(new User(id, email));
                emailGateway().send(email, "welcome " + id);
                return u;
            }
```

Solution (3)

```
            public String welcome(String id) {
                return userRepo().find(id)
                        .map(u -> "hello " + u.email())
                        .orElseThrow(() -> new IllegalArgumentException("missing:" + id));
            }
        };
    }
}
public interface UserControllerComponent extends UserServiceComponent {
    default UserController userController() {
        return new UserController() {
            public String postRegister(String id, String email) {
                try {
                    var u = userService().register(id, email);
                    return "201:" + u.id();
                } catch (IllegalStateException e) {
                    return "409:" + e.getMessage();
                }
            }
            public String getWelcome(String id) {
                try {
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Cake: repo → service → controller ===");
        System.out.println("--- naive ---");
        System.out.println(new NaiveApp().postRegister("u1", "a@b.c"));

        System.out.println("--- live cake ---");
        var live = new AppCake.LiveCake();
        System.out.println(live.userController().postRegister("u2", "x@y.z"));
        System.out.println(live.userController().getWelcome("u2"));
        System.out.println("mail=" + live.mailbox());
        System.out.println(live.userController().postRegister("u2", "x@y.z")); // conflict

        System.out.println("--- test cake with fakes ---");
        var test = new AppCake.TestCake().seed(new AppCake.User("seed", "s@e.ed"));
        System.out.println(test.userController().getWelcome("seed"));
        System.out.println(test.userController().postRegister("u3", "t@est.io"));
        System.out.println("mail=" + test.mailbox());
        System.out.println(test.userController().getWelcome("missing"));
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Cake: repo → service → controller ===
--- naive ---
email a@b.c
201:u1
--- live cake ---
201:u2
200:hello x@y.z
mail=[x@y.z:welcome u2]
409:exists:u2
--- test cake with fakes ---
200:hello s@e.ed
201:u3
mail=[t@est.io:welcome u3]
404:missing:missing
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Cake.

Related Patterns

- Contrast Cake with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Cake → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.